Expert Systems With Applications 238 (2024) 121865

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding

Zhenzhou Tian [a,b,c,*], Binhui Tian [a], Jiajun Lv [a], Yanping Chen [a,b,c], Lingwei Chen [d]

[a] School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[b] Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[c] Xi'an Key Laboratory of Big Data and Intelligent Computing, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[d] Department of Computer Science and Engineering, Wright State University, Dayton, 45435, OH, USA

ARTICLE INFO

Keywords:
Vulnerability detection
Tree decomposition
Tree-structured neural network
Deep semantic extraction

ABSTRACT

The explosive growth of software vulnerabilities poses a serious threat to the system security and has become one of the urgent problems of the day. However, existing vulnerability detection methods are still faced with limitations in reaching the balance between detection accuracy, efficiency and applicability. Following a divide-and-conquer strategy, this paper proposes TrVD (abstract syntax Tree decomposition based Vulnerability Detector) to disclose the indicative semantics implied in the source code fragments for accurate and efficient vulnerability detection. To facilitate the capture of subtle semantic features, TrVD converts the AST of a code fragment into an ordered set of sub-trees of restricted sizes and depths with a novel decomposition algorithm. The semantics of each sub-tree can thus be effectively collected with a carefully designed tree-structured neural network. Finally, a Transformer-style encoder is utilized to aggregate the long-range contextual semantics of all sub-trees into a vulnerability-specific vector to represent the target code fragment. The extensive experiments conducted on five large datasets consisting of diverse real-world and synthetic vulnerable samples demonstrate the performance superiority of TrVD against SOTA approaches in detecting the presence of vulnerabilities and pinpointing the vulnerability types. The ablation studies also confirm the effectiveness of TrVD's core designs.

1. Introduction

With the open source software (OSS) growing as one of the core parts of the software supply chain, the software security assurance becomes ever-more critical and challenging. The increasing dependencies (especially the multi-level transitive ones) on third-party libraries and the wide reuses of code snippets from social coding platforms in the developers' projects (Moore, 2022) have greatly accelerated the propagation of vulnerabilities potentially embedded within these publically-accessible codes. As of April 2023, the number of vulnerabilities disclosed on the CVE (Common Vulnerabilities and Exposures) website (CVE, 2023) has reached over 200,000, which exhibits a growing trend by years.

Considering ubiquitous vulnerability exploitation and the huge security threats it poses, it is thus of great essential for the developers to proactively discover the vulnerabilities implied in the code from either their own projects or the OSS to be integrated. However, due to that the security-related domain knowledge and experience required to identify the multifarious vulnerabilities generally go beyond the expertise of most developers, vulnerability detection represents significant challenges. It becomes increasingly impractical even for security professionals to manually find the potential vulnerabilities from millions of lines of code that reside in modern complex software systems, in view of massive time and efforts to put into.

With the rapid development in machine learning (ML), and especially the revolutionary learning structures and capabilities raised by deep learning (DL) (Li, Chen, & Wu, 2021), both academia and industry have started to exploit different data-driven learning techniques to automate the identification of vulnerabilities in the source code. Traditionally, code similarity-based methods (Kim, Woo, Lee, & Oh, 2017a; Li, Lu, Myagmar, & Zhou, 2006; Li et al., 2016; Wi, Woo, Whang, & Son, 2022) only detect cloned vulnerabilities via similarity comparison between the target code and the vulnerable code; static rule-based vulnerability detectors (Lipp, Banescu, & Pretschner, 2022) (e.g., Checkmarx (2022), Rats (Google, 2013), Coverity (Synopsys, 2022) and Flawfinder (A. Wheeler, 2014)) rely heavily on manually-defined rules abstracted by domain experts to scan the source code for specific types of vulnerabilities (e.g., Buffer Overflow, NULL Pointer

* Corresponding author at: School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China.
E-mail addresses: tianzhenzhou@xupt.edu.cn (Z. Tian), mx_info@stu.xupt.edu.cn (B. Tian), jjlv@stu.xupt.edu.cn (J. Lv), chenyp@xupt.edu.cn (Y. Chen), lingwei.chen@wright.edu (L. Chen).

https://doi.org/10.1016/j.eswa.2023.121865
Received 22 April 2023; Received in revised form 21 August 2023; Accepted 25 September 2023
Available online 30 September 2023
0957-4174/© 2023 Elsevier Ltd. All rights reserved.

Dereference). By contrast, learning-based vulnerability detection methods (Chakraborty, Krishna, Ding, & Ray, 2021; Lin, Wen, Han, Zhang, & Xiang, 2020) improve upon those widely-studied defect prediction techniques (Li, Wang, Nguyen, & Van Nguyen, 2019; Pradel & Sen, 2018; Wang, Wang, Nam, & Nagappan, 2021) and tend to detect more general vulnerabilities with less false positives/negatives and much less human efforts in summarizing vulnerability patterns.

Generally, these learning-based methods build classifiers based on the labeled training samples and the semantic-aware features extracted from them to map the target code snippets into the class space indicating either the absence/presence of vulnerabilities or specific vulnerability type, such that the follow-up repair of the identified vulnerability can be well guided. These features can be human-engineered metrics (e.g., code churn, complexity metrics, imports and function calls, developer-activity metrics) adopted by the conventional ML-based methods (Ghaffarian & Shahriari, 2017; Grieco et al., 2016; Morrison, Herzig, Murphy, & Williams, 2015), or semantic vectors encoded by representation learning models in the DL-based methods (Cheng, Zhang, Wang, & Sui, 2022; Wu et al., 2022; Zhou, Liu, Siow, Du, & Liu, 2019).

Despite their tremendous progresses, the emerging DL-based vulnerability detection methods (Tang, Hu, Hu, Kuang, & Chen, 2022) are still faced with certain limits in either detection accuracy, efficiency, or practical applicability. To address these limits, we propose a more practical abstract syntax Tree decomposition based Vulnerability Detector, called TrVD, to deeply delve the vulnerability semantics implied in the source code fragments so as to achieve accurate and efficient detection. Specifically, TrVD first constructs the AST of the target code fragment and converts it into an ordered set of sub-trees with a new decomposition algorithm. By operating on the sub-trees that correspond to fine-grained code units, TrVD is able to capture the subtle semantics from the code in an easier yet more precise way, which avoids expensive calculation and over-smoothed embedding over large/deep tree structures of the full AST using regular graph/tree neural networks. In addition, compared with other tree linearization methods (Alon, Zilberstein, Levy, & Yahav, 2019; Lin et al., 2018) that degrade AST to structure-less token sequences, the sub-trees produced by TrVD offer better expressive power for code units and enable more effective representation learning with proper neural networks. To this end, for each sub-tree, TrVD resorts to an attention-augmented tree-structured recursive neural network (RvNN) to distil its latent semantics.

Moreover, to further encode the interactions among these sub-trees and attend differentially to more and less important sub-trees when constructing vulnerability patterns, TrVD utilizes a Transformer-based backbone to aggregate the long-range contextual semantics of all sub-trees into a dense and vulnerability-specific numerical vector to represent the target code fragment. In summary, TrVD is empowered by the structured AST representation for any code, the light-weight code processing without reliance on particular code elements (e.g., library or API calls required by slice-based methods), as well as the sophisticated vulnerability semantic extraction using newly designed code decomposition and embedding paradigms, which lead to an accurate, efficient, and more practically applicable vulnerability detection approach. The evaluations conducted on five large datasets that consist of diverse synthetic and real-world vulnerable/non-vulnerable C/C++ functions well demonstrate TrVD's superiority in these aspects.

The main contributions of this paper are as follows:

- We propose a novel vulnerability detection method called TrVD, with elaborate design in every formulation phase, to make it an accurate, efficient and more practical approach that applies to code fragments. Specifically, to advance its ability in detecting well-hidden vulnerabilities and the specific vulnerability types, it combines novel AST decomposition, attention-augmented sub-tree encoding, and context-aware semantic aggregation to enable the distillation of subtle yet vulnerability-specific semantics from the target code fragments, which, to the best of our knowledge, is the first of this kind in the DL-based vulnerability detection works.
- We conduct extensive experiments to evaluate the performance of TrVD, the effectiveness of its different composing modules, and the runtime overheads. The empirical studies show that TrVD outperforms state-of-the-art DL-based methods on the majority datasets in terms of accuracy, F_1 and precision in either detecting the existence of vulnerability or identifying the specific vulnerability types. The superiority of TrVD's core designs, such as AST decomposition and sub-tree encoding, are also confirmed through ablation studies.
- We contribute a new dataset to facilitate vulnerability detection researches. This dataset contains 264,822 C/C++ functions, each of which is labeled with either specific CWE ID or non-vulnerable ground truth. The dataset and the source code for TrVD implementation have been made available at https://github.com/XUPT-SSS/TrVD to facilitate future benchmarking or comparison.

The rest of this paper is organized as follows. Section 2 discusses the background knowledge and the core of our approach. The related works are reviewed in Section 3. Section 4 details the designs of TrVD. Section 5 presents the experimental evaluations and analyses. Section 6 discusses possible threats to validity issues, the limitations and interesting future works we plan to extend. Finally, Section 7 concludes this work.

2. Preliminaries and the core

2.1. Basic terms and concepts

Vulnerability refers to the presence of weaknesses, defects, or security bugs within computer systems that can be potentially exploited by attackers. These vulnerabilities enable unauthorized access, such as stealing sensitive data, or allow attackers to execute arbitrary actions, such as installing malware, on the targeted computer system. Such vulnerabilities can manifest across diverse aspects, including software code, hardware components, configurations, or design. This study focuses on detecting potential vulnerabilities embedded in the code.

CWE, which stands for Common Weakness Enumeration, is a community-developed vulnerability list (CWE, 2023). It offers a standardized and structured way to identify and categorize these vulnerabilities, assigning a unique identifier to each one. For example, CWE-119 refers to the infamous "Buffer Overflow". Following different levels of conceptual abstraction, CWE organizes the vulnerabilities in a tree-like hierarchical structure, where low-level CWE-IDs are affiliated to higher-level CWE-IDs. For instance, CWE-787 that denotes "Out-of-bounds Write" and CWE-125 that denotes "Out-of-bounds Read", are both lower-level types affiliated with CWE-119.

We further provide brief descriptions of some commonly used code representations in program analysis tasks as follows.

Code tokens refer to lexical tokens in a program that carry specific semantics. They encompass identifiers (e.g., variable and function names), keywords, separators (e.g., punctuation and delimiters), and operators (e.g., arithmetic and logic operators). A code fragment can be straightforwardly represented as a sequence or set of tokens through lexical parsing.

AST, or Abstract Syntax Tree, is a fundamental code representation that organizes code elements into a tree structure. Tree leaves correspond to primary code elements, such as variable types, symbols, and operators, while non-leaf nodes represent a restricted set of code structures, such as expressions and loops (Tang, Shen et al., 2022). Compared to lexically parsed code tokens, AST also naturally manifests the syntactic structure of source code in addition to its lexical information.

CFG, or Control Flow Graph, is a directed graph where each node represents a basic block of statements, and each edge represents the flow of control between the blocks within a function. The CFG is constructed by identifying the control dependencies in the AST. It

generally requires the code to be functionally complete and compilable to precisely generate the CFG.

PDG, or Program Dependency Graph (Harrold, Malloy, & Rothermel, 1993), is another graph representation of the code that emphasizes both data and control dependencies between code elements. Similar to the CFG, it can be constructed on the basis of AST. During construction, certain code details are abstracted to reveal control and data dependencies more explicitly.

CPG, or Code Property Graph (Yamaguchi, Golde, Arp, & Rieck, 2014), presents a more synthesized hybrid graph representation of code, which integrates information derived from the AST, CFG and PDG.

2.2. Problem to solve

Various methods have been developed to detect vulnerabilities (Cui et al., 2022), including static, dynamic, concolic analysis, and fuzzing approaches. As we will delve into further in the related work section, TrVD falls into the learning-based paradigm within the static detection series. In this paradigm, the vulnerability detection task is formulated as a classification problem.

Specifically, in the training phase, a classifier $\mathcal{M}$ is learned from a set of training samples with ground-truth labels through code representation construction, feature extraction, and model training. In the detection phase, when presented with a potentially unseen piece of source code C, the same procedures of code representation construction and feature extraction are performed. The trained classifier $\mathcal{M}$ then predicts the presence of vulnerability, or further pinpoint the specific vulnerability type.

We aim to develop TrVD into an accurate, efficient and more practically applicable vulnerability detection method on source code. As the first training step, the selection of an appropriate initial code representation is essential to achieve the goals TrVD aims for. In this respect, TrVD resorts to operating on the AST, a choice that is believed to benefit TrVD from the following three major aspects.

- **Availability**. Other well-adopted code representations, as discussed in the previous section, include CFG (Cao, Sun, Bo, Wei, & Li, 2021; Cheng et al., 2019), PDG (Li, Wang, & Nguyen, 2021), and diverse graph-based variants (Chakraborty et al., 2021; Ding et al., 2022). These representations more explicitly delineate control or data dependencies among code elements, which, however, are challenging to be derived precisely when facing non-compilable or incomplete code fragments. Therefore, they may not always be feasible for vulnerability detection. By contract, AST can be constructed easily for any code fragment, e.g., a file, a function, or a single statement. Such wide availability ensures TrVD's practical applicability.
- **Efficiency**. Compared with code representations (e.g., CFG, PDG and code gadgets) (Li, Zou, Xu, Jin et al., 2022; Li et al., 2018; Tang, Hu et al., 2022; Zou, Wang, Xu, Li, & Jin, 2021) that require relatively complex and time-consuming control or dependency analysis, constructing an AST from code is much more straightforward and light-weight, thus contributing to the efficiency of the whole detection method.
- **Semantic Comprehensiveness**. Those artificially-created code representations (e.g., PDG and XFG (Cheng, Wang, Hua, Xu, & Sui, 2021)), tend to emphasize specific aspects of the code, such as the control flow or data dependency relationships (Harrold et al., 1993). However, they often lose some important information during the transformation process, which incurs semantic losses, particularly when representing incomplete code fragments. Differently, AST makes the highly structured nature of source code, where the underlying syntax regarding statements and expressions is directly available (Hou, Chen, & Ye, 2022); hence, the original code fragment can be exactly restored from it. In other words, AST provides more comprehensive, richer, and more precise code semantics and enables TrVD not to miss any suspicious vulnerability implications and enhance the detection accuracy.

However, as one may have noticed, despite the impressive properties of AST in facilitating DL-based vulnerability detection, its semantics are not so obvious as those purposefully-crafted representations such as CFG and PDG that intentionally make semantics more explicit. Thus, the major challenge faced by TrVD is how to comprehensively and effectively learn the suspicious semantics that are indicative of vulnerabilities from the expressive yet implicative AST.

2.3. The core

To fully exploit the potential of the AST and effectively address the challenge of its usage as previously discussed, which significantly impacts on vulnerability detection capability, TrVD follows the classic divide-and-conquer strategy.

Divide. Encoding the entire AST using graph/tree-based neural networks, such as GCN (Kipf & Welling, 2016), GAT (Veličković et al., 2017), Tree-LSTM (Tai, Socher, & Manning, 2015), and TBCNN (Mou, Li, Zhang, Wang, & Jin, 2016), may greatly degrade in capturing the long-range dependencies with over-smoothed embedding when dealing with large/deep tree structures (Zhang et al., 2019). Due to resource restrictions, it is also highly infeasible for these time- and memory-consuming models to handle such large/deep ASTs directly. In this regard, TrVD opts to divide the whole AST into an ordered set of sub-trees via a decomposition algorithm, each of which corresponds to a fine-grained intact code unit in the original code fragment. This yields two advantages here: (1) each sub-tree contains a limited (much smaller) number of nodes and controllable tree depth, and can be manipulated by general tree or graph neural networks in a cost-inexpensive way; and (2) more importantly, this offers an opportunity to better grasp the subtle semantics indicating vulnerability information by focusing on a local code unit of restricted size, which otherwise may be easily overlooked.

Conquer. TrVD adopts a "comprehensive-acquisition followed by critical-point-focus" scheme to achieve the effective extraction of indicative and suspicious vulnerability semantics. To be specific, TrVD first processes each sub-tree with a newly designed tree-structured neural network to map the semantics of its corresponding code unit into a numerical vector. Based on the learned sub-tree embeddings, TrVD then leverages a Transformer-based backbone to attend differentially to more and less important sub-trees to discover vulnerability patterns with self-attention mechanism, and fuse the long-range contextual semantics of all sub-trees into a dense and vulnerability-specific numerical vector to represent the target code fragment. In this sense, the way of TrVD in code representation learning is similar to slice-based methods (Li, Zou, Xu, Chen et al., 2022; Li, Zou, Xu, Jin et al., 2022; Li et al., 2018; Zou et al., 2021), where code gadgets generated by slicing the relevant statements according to points of interest (e.g., library/API functions) accommodate information useful for learning local features and helping pinpoint vulnerability patterns through regular NLP models (e.g., LSTM). But sub-trees refined in TrVD are more structured and less ambiguous, while the Transformer devised for attention-augmented sub-tree aggregation enables better contextualized embedding to improve vulnerability detection performance.

3. Related work

We focus on reviewing the closely-related static source code vulnerability detection works that fall into the following categories, including code similarity matching based methods, static rule based methods and learning based methods.

3.1. Similarity matching-based methods

This line of methods (Cui, Hao, Jiao, Fei, & Yun, 2021; Jang, Agrawal, & Brumley, 2012; Kang, Son, & Heo, 2022; Kim, Woo, Lee, & Oh, 2017b; Pham, Nguyen, Nguyen, & Nguyen, 2010) generally calculate the similarity between the abstracted representations of the target code and the code of known vulnerability, to determine the absence/presence of potential vulnerabilities within the target code. The specific code representations adopted by existing approaches for similarity matching are varying, such as tokens (Jang et al., 2012; Kim et al., 2017b), trees (Pham et al., 2010; Yamaguchi, Lottmann, & Rieck, 2012), graphs (Cui et al., 2021), their hybrids (Li et al., 2016), well-crafted signatures (Kang et al., 2022; Xiao et al., 2020) and embeddings (Sun et al., 2021) learned by siamese neural networks. Overall, they excel at detecting recurring vulnerabilities introduced by code or library reusing, rather than general or undisclosed vulnerabilities.

3.2. Static rule-based methods

The static rule-based methods proceed by scanning the target source code with a mass of elaborately defined vulnerability rules or patterns. The typical static analyzers, such as FlawFinder (A. Wheeler, 2014), Cppcheck (Marjamaki, 2013), Infer (Infer, 2013), CodeChecker (CodeChecker, 2013) and Checkmarx (Checkmarx, 2022) fall into this category. However, the rules used to scan the code are mainly summarized by experienced security experts, with each of them describing a specific vulnerability scenario. Thus, as confirmed by existing studies (Goseva-Popstojanova & Perhinschi, 2015; Lipp et al., 2022), this line of methods are only effective for vulnerabilities of certain types (e.g., buffer-overflows, use-after-frees) and tend to report a large portion of false positives or false negatives in detecting vulnerabilities due to the diversity of the real-world code and possible vulnerability patterns. Recent researches (Cheng et al., 2021; Li, Zou, Xu, Jin et al., 2022; Wu et al., 2022) that follow the learning-based roadmap show superior vulnerability detection performance over the rule-based methods.

3.3. Learning-based methods

The early works (Ghaffarian & Shahriari, 2017; Perl et al., 2015; Younis, Malaiya, Anderson, & Ray, 2016) generally adopt traditional machine learning algorithms to train detection models with the representative features summarized by experts, such as code complexity metrics, code churns, imports and calls, and developer activities (Bosu, Carver, Hafiz, Hilley, & Janni, 2014). As a whole, these engineered features are inadequate in indicating the presence of vulnerabilities. Also, most existing methods are limited to the in-project rather than the general-purpose vulnerability detection.

With great successes of deep learning in a wide range of applications, the recent works that achieve SOTA vulnerability detection performances all employ deep learning schema. Resting on the initial code representations that the neural models operate on, existing DL-based methods can be generally summarized into three categories: token-based methods, slice-based methods and graph-based methods. The token-based methods convert the code into one or a set of token sequences by lexically parsing the code (Russell et al., 2018) or linearizing its abstract syntax tree (Lin, Zhang, Luo, Pan, & Xiang, 2017; Lin et al., 2018) in certain manners (e.g., pre-order traversal, leaf-node path traversal with sampling) so as to preserve additional contextual and structural information. They are efficient due to the light-weight code processing and the low-overhead code encoding with sequence-oriented neural network structures, but generally incapable of accurately detecting well-hidden vulnerabilities. The slice-based methods (Li, Zou, Xu, Chen et al., 2022; Li, Zou, Xu, Jin et al., 2022; Li et al., 2018; Zou et al., 2021) represent the code as code gadgets (Li et al., 2018; Tang, Hu et al., 2022; Zou et al., 2021) (i.e., groups of statements that have data or control dependencies) or the derivatives (e.g., SeVCs (Li, Zou, Xu, Jin et al., 2022), iSeVCs (Li, Zou, Xu, Chen et al., 2022)). They are inefficient for enforcing the time-consuming data and control dependency analysis as well as performing slicing; furthermore, as the slices are generated against certain code elements (i.e., program points of interest, such as the library/API function calls, the array usages, etc.), these methods become inapplicable in detecting vulnerabilities that are irrelevant to these elements. The graph-based methods (Cheng et al., 2021; Li, Wang et al., 2021) encode with graph neural networks (Shi et al., 2022; Wu et al., 2021) the widely-used conventional graph-structured code representations, such as CFG (Cao et al., 2021; Cheng et al., 2019), PDG (Li, Wang et al., 2021), their hybrids (Zhou et al., 2019), or elaborately derived structures, such as CPG (Chakraborty et al., 2021; Ding et al., 2022) and XFG (Cheng et al., 2021). They are more sophisticated and time-consuming in either constructing these graph structures or training well-behaving models. Compared with these methods, our TrVD aims to reach and has indeed achieved a good balance between detection accuracy, efficiency and applicability.

To provide a clearer insight into the distinctiveness of TrVD in contrast to the aforementioned DL-based approaches, Table 1 presents a summary of their fundamental characteristics. This includes their major categories, the initial code representations they work with, the processing employed to transform the initial representation, the final format for model input after transformation, the applied learning model, and a qualitative assessment of the computational overhead.

4. Design of TrVD

Fig. 1 depicts the overall architecture of TrVD, which involves the training phase that learns a classifier $\mathcal{M}$ on the vulnerability datasets with well-labeled ground truths (which will be discussed in Section 5.1.1), and the detection phase that uses the trained classifier $\mathcal{M}$ to output a prediction. During both phases, the target source code fragment is fed through five main modules in turn to get processed: (1) AST Generator, which normalizes the code fragment in a semantic-preserving fashion and constructs its AST with a parser; (2) AST Decomposer, which converts the AST into an ordered set of subtrees with a decomposition algorithm; (3) Comprehensive Semantic Collector, which iteratively distils the conveyed semantics from each sub-tree into a real-valued vector with a tree-structured neural encoder; (4) Suspicious Semantic Focuser, which aggregates all the collected subtree semantics into a contextualized embedding vector that attends to more vulnerability-specific semantics with Transformer-based model; (5) Vulnerability Detector, which implements a multi-layer perceptron (MLP) with softmax layer trained with cross-entropy loss to predict an output. The details of these five modules are discussed as follows.

4.1. AST generator

Instead of directly constructing the AST for the original source code fragment, we opt to perform light-weight syntactic abstractions that normalize its code tokens with the following considerations. First, different coding habits naturally present among different programmers, where their personalized naming styles for function and variable names barely indicate any vulnerability information, but may act as noises to deteriorate the code representation learning by immersing TrVD in too many unnecessary details. Second, to facilitate the distillation of semantics from the sub-trees with neural encoders, each tree node that corresponds to a certain token needs to be pre-mapped into embedding space; however, retaining all the original tokens may raise two issues: (1) enlarging the vocabulary size that accordingly increases the embedding complexity and decreases the embedding quality; (2) intensifying the out-of-vocabulary (OOV) impact and degrading subsequent vulnerability-indicative semantics extraction.

Table 1
Major characteristics of the DL-based vulnerability detection approaches.

Category	Approaches	Initial representation	Processing	Model inputs	Model	Overhead
Token-based	Russell et al. (2018)	Code	Tokenization	Token sequence	CNN	Low
	Lin et al. (2017)	AST	Serialization	Token sequence	BiLSTM	Low
	Lin et al. (2018)	AST	Serialization	Token sequence	BiLSTM	Low
Slice-based	Li et al. (2018)	DDG	Slicing	Code Gadgets	BiLSTM	High
	Tang, Hu et al. (2022)	PDG	Slicing	Code Gadgets	CNN+Attention	High
	Zou et al. (2021)	PDG	Slicing	Code Attention and Gadgets	BiLSTM	High
	Li, Zou, Xu, Jin et al. (2022)	PDG	Slicing	SeVCs	BiGRU	High
	Li, Zou, Xu, Chen et al. (2022)	PDG	Slicing	iSeVCs	BiRNN+Attention	High
Graph-based	Cheng et al. (2019)	CFG	–	CFG	GCN	High
	Li, Wang et al. (2021)	PDG	–	PDG	FA-GCN	High
	Zhou et al. (2019)	CPG	–	CPG	GGNN	High
	Chakraborty et al. (2021)	CPG	–	CPG	GGNN	High
	Cheng et al. (2021)	PDG	Slicing	XFG	GNN	High
	Ding et al. (2022)	CPG	–	CPG	Transformer + GGNN	High
Tree-based	TrVD	AST	Decomposition	Sub-Tree Sequence	AttRvNN + Transformer	Medium

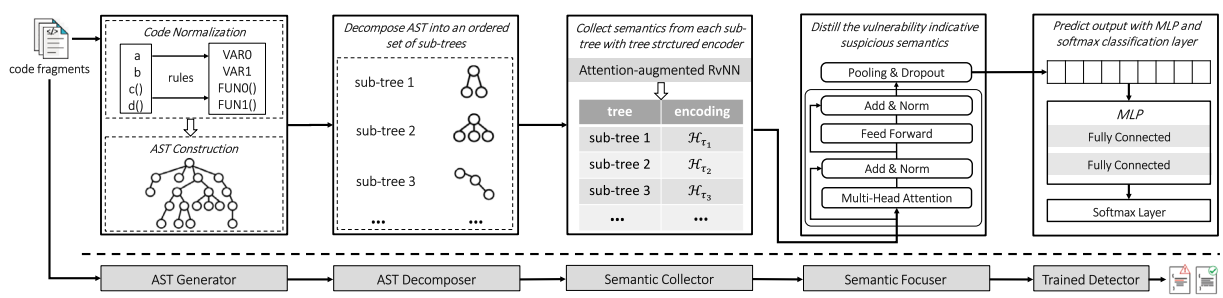

Fig. 1. The overview of TrVD.

More specifically, after removing the comment lines from the original code, we normalize its remaining code tokens, by reassigning each user-defined variable and function with a new symbolic name in the form of *VAR_i* and *FUN_i* respectively, where i is the order of the variable or function for its first appearance in the code fragment, while the remaining tokens, including reserved words, literals, library function names, and punctuations remain intact.

Fig. 2 illustrates a normalization processing example that is performed on a short code snippet. It should be noted that the normalization performed according to the above defined abstraction rules is semantic-preserving; that is, no semantic losses are enforced during the normalization phase, which is important to ensure the reliability of the whole method. On the basis of the normalized code fragment, we then leverage tree-sitter,[1] a widely-adopted C/C++ parsing library, to obtain its abstract syntax tree.

4.2. AST decomposer

As discussed in Section 2.3 about the problems raised by encoding the AST as a whole for vulnerability detection, this module decomposes the AST into finer-grained code units that are much more affordable to subsequent semantic encoders, and make the subtle semantics conveyed in the code more explicit to the neural encoders. Formally, given the AST $\mathcal{T}$, the task of our decomposer is to divide it into an ordered set $\mathcal{D}$ with a policy ζ, where $\mathcal{D}$ is composed of code units of restricted sizes.

Different policies have been designed to decompose the AST to facilitate source code analysis tasks, which can be summarized into three major categories according to their outputs.

- Single token sequence. Traversal algorithms are generally utilized to flatten the AST into a token sequence, including pre-order traversal (Tang, Shen et al., 2022; Zhang, Wang, Zhang, Sun, & Liu, 2020) and in-order traversal (Svyatkovskiy, Zhao, Fu, & Sundaresan, 2019). To avoid information loss, SBT (Hu, Li, Xia, Lo, & Jin, 2018) and SPT (Niu et al., 2022) further improve these methods by appending additional symbols to indicate the parent–child relationships to ensure that the linearized sequence can be unambiguously mapped back to the AST, which, however, complicates the sequence with larger size. Another method code2seq (Alon, Levy, & Yahav, 2019) concatenates the sampled paths collected between the leaf nodes to form a single token sequence, which is often too long to be efficiently handled by the neural encoders for semantic extraction, especially for vulnerability detection task.
- Token sequence set. PathTrans (Kim, Zhao, Tian, & Chandra, 2021) maps the AST to a set of root-paths, each of which is comprised of the tokens obtained by traversing either up from a leaf node to the root, or down from the root to a leaf node (Jiang, Zheng, Lyu, Li, & Lyu, 2021), and points out the longitudinal context that a specific code element (i.e., the leaf node) resides in. The root-paths can serve as inputs for learning token embeddings; however, as each path only contains fragmentary code elements, the incomplete semantics learned from them may degrade the training and the detection performance.
- Sub-trees. Instead of linearizing the AST as token sequences, methods such as ASTNN (Zhang et al., 2019) and Infercode (Bui, Yu, & Jiang, 2021) decompose it into a set of sub-trees to leverage the syntactic structures. Different granularities are adopted to split the AST, where ASTNN produces non-overlapping sub-trees that correspond to code statements, while Infercode produces sub-trees with overlaps that correspond to smaller code elements such as an expression.

[1] https://github.com/tree-sitter/tree-sitter

```
1: void bad()
2: {
3:   wchar_t * data;
4:   wchar_t dataBadBuffer[10];
5:   wchar_t dataGoodBuffer[10+1];
6:   if(GLOBAL_CONST_TRUE)
7:   {
8:     /* FLAW: Set a pointer to a buffer that does not leave room for a NULL
9:                 terminator when performing string copies in the sinks  */
10:      data = dataBadBuffer;
11:      data[0] = L'\0'; /* null terminate */
12:  }
13:  {
14:      wchar_t source[10+1] = SRC_STRING;
15:      size_t i, sourceLen;
16:      sourceLen = wcslen(source);
17:      /* Copy length + 1 to include NUL terminator from source */
18:      /* POTENTIAL FLAW: data may not have enough space to hold source */
19:      for (i = 0; i < sourceLen + 1; i++)
20:      {
21:         data[i] = source[i];
22:      }
23:      printWLine(data);
24:  }
25: }
```

```
1: void FUN0()
2: {
3:   wchar_t * VAR0;
4:   wchar_t VAR1[10];
5:   wchar_t VAR2[10+1];
6:   if(GLOBAL_CONST_TRUE)
7:   {
10:      VAR0 = VAR1;
11:      VAR0[0] = L'\0';
12:  }
13:  {
14:      wchar_t VAR3[10+1] = SRC_STRING;
15:      size_t VAR4, VAR5;
16:      VAR5 = wcslen(VAR3);
19:      for (VAR4 = 0; VAR4 < VAR5 + 1; VAR4 ++)
20:      {
21:         VAR0[VAR4] = VAR3 [VAR4];
22:      }
23:      FUN1(VAR0);
24:  }
25: }
```

Fig. 2. An normalization processing example that is performed on a code snippet from our dataset.

Our decomposer follows a similar policy as ASTNN to divide the AST into a sequence of non-overlapping sub-trees at the granularity of statement, but dismisses the finer-grained expression or single-token sub-tree used in Infercode. The reasons behind this are that (1) the latter method generates a very long sub-tree sequence where the long-range context weakens the intrinsic relationships among sub-trees, and (2) each sub-tree can only accommodate incomplete or even useless code information. Also, ASTNN always splits a compound statement (e.g., *if* statement) into a header sub-tree and a set of sub-trees that corresponds to the statements within its body, which disrupts the local semantic and structural integrity. Different from ASTNN, our decomposer only decomposes these compound statements when necessary. Specifically, we set up a checker to inspect the size and depth of the sub-tree rooted at each compound statement node, and further split it only when either its size or depth exceeds certain threshold. As such, a good balance can be reached by producing sub-trees with light weight yet preserving relatively complete local semantics.

Algorithm 1 illustrates the detailed processing steps of our decomposer. Given the root node, it proceeds by recursively splitting out sub-trees through a visitor and a constructor. The visitor performs pre-order traversal along the AST to pass each node to the constructor for node type inspection, compound statement decomposition, and sub-tree construction, where the constructed sub-trees are appended to the set in order. Here, the order for each sub-tree in the final set is decided by the position of its corresponding code in the original code fragment. An illustrative example is given in Fig. 3 to facilitate the understanding of our AST decomposition algorithm as well as its differences from ASTNN. More specifically, taking the normalized code in Fig. 3(a) as input, our algorithm decomposes its AST into 12 sub-trees ordered by their appearances in the code. Compared with ASTNN, the differences of our algorithm are also illustrated: for the *if* compound statement from line 6 to line 10 in the code, our algorithm generates one integrated sub-tree, while ASTNN splits it into three trivial sub-trees for each sub-statement as framed by the blue dotted lines in Fig. 3(a).

Algorithm 1 Decomposing AST into Statement Sub-trees

Input:
  $\mathcal{T}$: an abstract syntax tree.
  α: the maximum depth allowed for a sub-tree.
  β: the maximum number of nodes allowed in a sub-tree.
Output:
  $\mathcal{D}$: an ordered sub-tree set.
  function TreeSplitting(τ, $\mathcal{D}$, α, β, $\mathcal{C}$)
    node ← getRoot(τ) ▷ get the root node of current tree
    if *node* is a statement **then**
      if *node* ∈ $\mathcal{C}$ **then**
        if trSize(*node*)< α and trDepth(*node*) < β **then** ▷ construct a sub-tree rooted at *node* and append it to $\mathcal{D}$
          t ← subTreeConstructor(*node*)
          $\mathcal{D} \leftarrow \mathcal{D} \oplus t$
        else
          t ← subTreeConstructor(*node.header*)
          $\mathcal{D} \leftarrow \mathcal{D} \oplus t$
          for each child node *child* of *node.body* **do**
            TreeSplitting(child, $\mathcal{D}$)
          end for
        end if
      else
        t ← subTreeConstructor(*node*)
        $\mathcal{D} \leftarrow \mathcal{D} \oplus t$
      end if
    else
      for each child node *child* of *node* **do**
        TreeSplitting(child, $\mathcal{D}$)
      end for
    end if
  end function
  $\mathcal{D} \leftarrow \emptyset$
  $\mathcal{C}$ ← [‘FuncDef’, ‘If’, ‘For’, ‘While’, ‘Dowhile’, ‘Switch’, ‘Try’, ‘Catch’]
  ▷ the type list of the compound statements
  TreeSplitting($\mathcal{T}$, $\mathcal{D}$, α, β, $\mathcal{C}$)
  return $\mathcal{D}$

4.3. Comprehensive semantic collector

Given the ordered sub-tree set $\mathcal{D} = \{\tau_1, \tau_2, \ldots, \tau_n\}$, where n denotes the number of sub-trees produced with the decomposer and τ_i denotes the ith sub-tree, the collector iteratively processes each of them with a tree-structured recursive neural network (RvNN) to collect its latent semantics. This follows a bottom-up style, where RvNN recursively aggregates the local node representations from the leaves all the way up to the root to derive a global representation for the entire sub-tree. Formally, let $\mathcal{S}_j$ denote the direct children of node v_j in the sub-tree

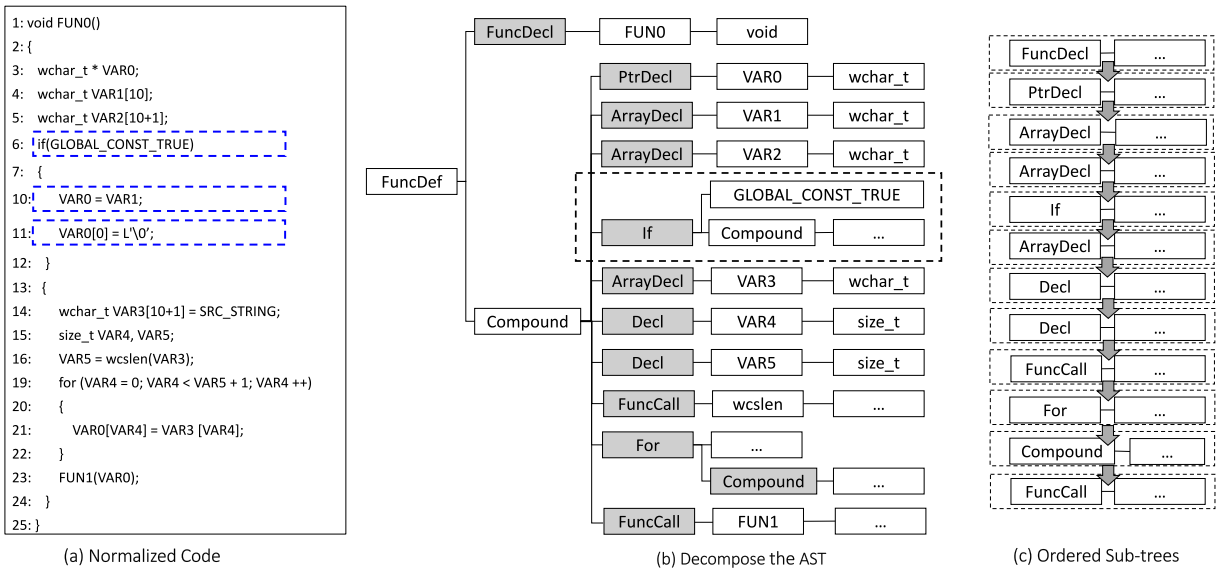

Fig. 3. An illustrative example of our AST decomposition on the normalized code snippet. The sub-figure (b) shows the AST as well as the split sub-trees, with the gray boxes marking out their roots. The difference of our decomposition algorithm compared with ASTNN is also illustrated, where our algorithm generates for the *if* compound statement from line 6 to line 10 one integrated sub-tree (marked with the dashed box in the sub-figure (b)), while ASTNN splits it into three trivial sub-trees for each sub-statement as framed by the blue dotted lines.

$\tau_i = \{V, E\}$, where V and E are its node set and edge set, respectively. The hidden state of node v_j is computed by aggregating the semantics of its child nodes and itself through a GRU-style neural unit, which can be formulated as:

$$\mathbf{r}_j = \sigma\left(\mathbf{W}_r\mathbf{h}_S + \mathbf{U}_r\mathbf{e}_j + \mathbf{b}_r\right), \tag{1}$$

$$\mathbf{z}_j = \sigma\left(\mathbf{W}_z\mathbf{h}_S + \mathbf{U}_z\mathbf{e}_j + \mathbf{b}_z\right), \tag{2}$$

$$\widetilde{\mathbf{h}}_j = tanh\left(\mathbf{W}_h\left(\mathbf{r}_j \odot \mathbf{h}_S\right) + \mathbf{U}_h\mathbf{e}_j + \mathbf{b}_h\right), \tag{3}$$

$$\mathbf{h}_j = \mathbf{z}_j \odot \mathbf{h}_S + \left(1 - \mathbf{z}_j\right) \odot \widetilde{\mathbf{h}}_j, \tag{4}$$

where σ is the sigmoid function; $\odot$ denotes the element-wise product, the **W**s, **U**s and **b**s are the weights and biases to be jointly learned with other parameters during the training process; $\mathbf{e}_j$ is the embedding vector that corresponds to the token in node v_j, which is learned with skip-gram (Mikolov, Chen, Corrado, & Dean, 2013) algorithm on a corpus of paths collected from the ASTs[2] to enable a higher initialization for training than straightforward yet computationally intractable one-hot embedding schema; $\mathbf{h}_S$ denotes the children's aggregated semantics by pooling the hidden states of all the nodes in $\mathcal{S}_j$.

There are multiple ways to pool the children's hidden states into a single vector $\mathbf{h}_S$, such as direct sum, average or max pooling. However, these pooling operations generally overlook the differences among the child nodes in the aspect of their semantics. In order to capture such semantic differences, we introduce an attention mechanism to strengthen hidden state aggregation over h_S with more expressive power, which allows the pooling process to pay more attention on more informative child nodes. Specifically, we adapt the context/query attention (Yang et al., 2016) to calculate an importance weight for each child node $s \in \mathcal{S}_j$, on the basis of which a weighted sum of the children's hidden states is computed as their aggregated semantics $\mathbf{h}_S$. This attention-augmented calculation can be formalized as:

$$\mathbf{h}_S = \sum_{s \in \mathcal{S}_j} \alpha_s \cdot \mathbf{h}_s, \tag{5}$$

$$\alpha_s = \frac{\exp\left(\boldsymbol{\beta}_s^T \boldsymbol{\mu}_w\right)}{\sum_{k \in \mathcal{S}_j} \exp\left(\boldsymbol{\beta}_k^T \boldsymbol{\mu}_w\right)}, \tag{6}$$

$$\boldsymbol{\beta}_s = \tanh\left(\mathbf{W}_s\mathbf{h}_s + \mathbf{b}_s\right) \tag{7}$$

where α_s is the normalized attention weights, which indicates the relative importance of each node and essentially controls how much information can flow from each child node to its parent node; $\mathbf{h}_s$ is the hidden state of node $s \in \mathcal{S}_j$; $\mathbf{W}_s$ and $\mathbf{b}_s$ are the respective weight matrix and bias vector corresponding to the hidden state $\mathbf{h}_s$, both of which can be jointly learned with the context/query vector $\boldsymbol{\mu}_w$ during the training process.

After iteratively calculating the hidden state of every node in the sub-tree using the attention-augmented tree-structured RvNN in a bottom-up way, we take the hidden state of its root node as the sub-tree's final semantic encoding vector. In this regard, for the sub-tree τ_i, its semantic encoding can be denoted as $\mathbf{h}_{\tau_i} = \mathbf{h}_{v_0} = \mathcal{H}(\tau_i)$, where $\mathbf{h}_{v_0}$ is the hidden state of the root node in sub-tree τ_i, and $\mathcal{H}(\cdot)$ is the semantic collector with composite functions presented in the Eqs. (4)~(7).

4.4. Suspicious semantics focuser

Based on the semantic encodings $\mathbf{H}_D = \{\mathbf{h}_{\tau_1}, \mathbf{h}_{\tau_2}, \ldots, \mathbf{h}_{\tau_n}\}$ systematically gathered from the ordered sub-tree set D by the collector, TrVD further utilizes the Focuser to aggregate them to obtain the semantic encoding $\mathbf{h}_{\mathcal{T}}$ for the entire AST $\mathcal{T}$, with the help of the Transformer architecture. It is well known that Transformer is currently the most prominent neural network architecture that achieves state-of-the-art performance against other sequence-oriented neural networks, such as the RNN, LSTM and TextCNN, in many NLP and program analysis tasks. By taking advantage of its multi-head self-attention mechanism, TrVD is able to capture the long-range contextual semantic relationships among the sub-trees, and pay more attention on the vulnerability-specific semantics.

The Focuser only uses the encoder part of the Transformer architecture to produce the AST's encoding $\mathbf{h}_{\mathcal{T}}$. More specifically, the sub-tree

[2] The root-paths as described in Section 4.2 are collected by traversing each AST in our datasets down from the root node to each of its leaf nodes that form a corpus of token sequences to be fed into the skip-gram algorithm for token embedding learning. Other alternative path collection ways, such as traversing from each leaf node to the root (Kim et al., 2021) or sampling the paths between the leaf nodes (Alon, Zilberstein et al., 2019), as well as other word embedding methods such as fastText (Joulin, Grave, Bojanowski, & Mikolov, 2017) and GloVe (Pennington, Socher, & Manning, 2014), can also be used to substitute our token embedding schema.

representation vectors $\mathbf{H}_D$ are first fed into a series of vertically stacked Transformer blocks; each block comprises a multi-head attention layer, a normalization layer and a position-wise feed-forward layer, which are used for contextual semantic relationship capturing and vulnerability-indicative semantic focusing. To implement the multi-head attention layer, the core operations are the self-attention score calculations, which can be computed within each head with the following equations:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathbf{T}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{8}$$

$$\mathbf{Q} = \mathbf{W}^Q\mathbf{H}_D,\ \mathbf{K} = \mathbf{W}^K\mathbf{H}_D,\ \mathbf{V} = \mathbf{W}^V\mathbf{H}_D \tag{9}$$

where $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{n\times d_v}$ is the attention score matrix, in which each row g_i is a d_v-dimensional attention vector for the corresponding row in the input matrix $\mathbf{H}_D$; $\mathbf{Q} \in \mathbb{R}^{l\times d_k}$, $\mathbf{K} \in \mathbb{R}^{m\times d_k}$ and $\mathbf{V} \in \mathbb{R}^{m\times d_v}$ are three matrices derived by multiplying the input $\mathbf{H}_D$ with trainable weight matrices; and $\sqrt{d_k}$ is a scaling factor to ensure more stable gradients by avoiding producing too large softmax results. After that, the distilled semantic vectors (i.e., the outputs of the stacked Transformer blocks) corresponding to the input sequence $\mathbf{H}_D$ at different time steps, are manipulated with the max-pooling operation to obtain the dense vector $\mathbf{h}_\mathcal{T}$ to represent the target code fragment.

4.5. Vulnerability detector

By taking the vector representation $\mathbf{h}_\mathcal{T}$ of the entire AST as input, the Detector module predicts the vulnerable/non-vulnerable result or the specific vulnerability type, with a multi-layer perceptron. The predicted probability distribution $\text{MLP}_\theta\left(\mathbf{h}_\mathcal{T}\right)$ is computed using a softmax function to facilitate minimizing the cross-entropy loss between the prediction and ground truth, where θ are the parameters from all modules that are optimized during the model training.

5. Experiments and evaluations

To evaluate TrVD, we seek to answer the following *Research Questions* (RQs):

- RQ1: **Performance on Vulnerability Presence Detection**. What is the performance of TrVD in detecting the presence of vulnerabilities against other methods?
- RQ2: **Performance on Vulnerability Type Detection**. What is the performance of TrVD in pinpointing the type of vulnerabilities compared with other methods?
- RQ3: **Ablation Study on TrVD's Core Components.** How do our designs including AST decomposition, sub-tree encoding and AST encoding contribute to the performance of TrVD?
- RQ4: **Efficiency Analysis of TrVD.** How about the runtime overhead of TrVD against other vulnerability detection methods?

5.1. Experimental setup

5.1.1. Datasets

To evaluate our method, we construct a dataset consisting of C/C++ functions on the basis of the Software Assurance Reference Dataset (SARD),[3] a vulnerability database that is widely used as the source for producing experimental samples. The programs in SARD are collected as a mixture of the production, academic and synthetic code, with each program associated with a "bad", "good" or "mixed" label. Each "bad" program generally contains one vulnerable function, while each "good" program only contains non-vulnerable functions that are fixed or patched from their vulnerable versions. Each "mixed" program organizes both the vulnerable function and its patched versions within one program.

[3] https://samate.nist.gov/SARD

Table 2
Some statistical information of the datasets. The usage scenario column depicts whether TrVD will be evaluated on them for vulnerable presence detection or further vulnerability type detection.

Dataset	#Vulnerable	#Non-Vulnerable	Dataset type	Usage scenario	
				Presence	Type
Ours	98,181	166,641	Synthetic	✓	✓
D2A	2795	2444	Real-World	✓	✗
VulDeePecker	9739	150,409	Hybrid	✓	✗
Reveal	2240	20,494	Real-World	✓	✗
muVulDeePecker	43,119	138,522	Hybrid	✗	✓

Based on the raw C/C++ programs in SARD, we first process them with the tree-sitter parsing library to identify and collect the vulnerable and their patched functions, and remove the ones that fail in converting into ASTs with our AST Generator. Then, each collected function is associated with its vulnerable/non-vulnerable ground truth, and each vulnerable function is further associated with a CWE-ID (Common Weakness Enumeration IDentifiers) (CVE, 2023) to indicate the specific vulnerability type it contains, based on the name of the file that each function resides in. As a result, we obtain a dataset consisting of 264,822 well-labeled functions, where 166,641 are non-vulnerable and 98,181 are vulnerable across 118 CWE-IDs.

The programs in SARD are basically synthetic. Thus, we also evaluate TrVD on four other vulnerability datasets that are used in the comparison works, including:

- **D2A** (Zheng et al., 2021), a real-world vulnerability dataset curated from several open-source software projects, such as FFmpeg, httpd, Libav, LibTIFF, Nginx and OpenSSL, by the IBM Research team. A differential analysis technique is designed to label the samples.
- **VulDeePecker** (Li et al., 2018), a hybrid dataset that consists of real-world samples from the National Vulnerability Database (NVD) and synthetic samples from the SARD.
- **Reveal** (Chakraborty et al., 2021), a real-world dataset curated by tracking the past vulnerabilities from the Linux Debian Kernel and Chromium.
- **muVulDeePecker** (Zou et al., 2021), a remarkably similar dataset as VulDeePecker which is also curated from the NVD and the SARD, but with the vulnerable samples distributed to 40 different vulnerability types.

In our experiments, the first three datasets which correlate each of their samples with a vulnerable or non-vulnerable ground truth, are used in the vulnerability presence detection task; while the muVulDeePecker dataset which contains well-labeled vulnerability type information is used in the vulnerability type detection task. Our constructed dataset is used for both tasks. A statistical summary on these datasets regarding quantities of their vulnerable and non-vulnerable samples as well as their usage scenarios in our evaluations, is reported in Table 2.

5.1.2. Implementation and experimental settings

We implement TrVD fully in python. Its AST Generator uses the widely-adopted tree-sitter for extracting ASTs from the code fragments written in the C/C++ programming language.[4] The default parameters α and β that guide the AST Decomposer to split out sub-trees are set to 8 for tree depth and 40 for tree size, respectively. The skip-gram algorithm for learning the nodes' initial token embeddings adopts the

[4] Other programming languages can be well-supported by substituting the C/C++ parser with matching parsers, as well as reorganizing the extracted AST into the supported format acceptable to TrVD's AST Decomposer.

implementation provided in gensim,[5] with the embedding dimension set to 128. The rest neural network modules are implemented with PyTorch, where the dimension of the GRU hidden state in the Collector is set to 128, and the hidden dimension of the Transformer block in the Focuser is set to 128 as well.

For the experimental settings, we split the datasets into training, validation and testing sets with the proportion of 8:1:1. Since the ratios between the vulnerable and non-vulnerable samples in some of the datasets are fairly imbalanced, we undersample the non-vulnerable samples to the quantity of vulnerable samples within each dataset, to maintain a balanced training set. As for the testing and validation sets, we use the real ratios presented in the original datasets. We train our models with an initial learning rate $1e-3$ (which is decreased by 0.8 every 10 epochs), a batch size of 32, and the Adam optimizer. In each epoch, the training samples are shuffled and accuracy on the validation set is calculated. The early stopping mechanism is leveraged to stop the training when the validation accuracy no longer rises for 5 epochs. Finally, the model that shows the best accuracy is taken as the final detection model, with which the performance evaluation results are reported on the testing set. All the experiments are conducted on a Linux server, with two 2.1 GHz Intel Xeon Silver-4310 CPUs, 128 GB RAM and two NVIDIA RTX3090 GPUs.

5.1.3. Evaluation metrics

Following most of existing learning-based vulnerability detection methods, we also use Accuracy, Precision, Recall and F_1-score as the metrics, for fair performance comparison against other methods. Note that, in the vulnerability type detection experiments, accuracy refers to total accuracy, while precision, recall and F_1-score refer to weighted-average precision, recall and F_1-score respectively.

5.2. Experimental results

5.2.1. RQ1: Performance on vulnerability presence detection

In this experiment, we evaluate the performance of TrVD in detecting the existence of vulnerability. State-of-the-art DL-based vulnerability detection methods, including VulDeePecker, SySeVR, VulBERTa, Devign and Reveal are used for performance comparison with TrVD. A brief introduction of the baseline methods is provided as follows:

- **VulDeePecker** (Li et al., 2018) and **SySeVR** (Li, Zou, Xu, Jin et al., 2022) are two slice-based methods, which feed code gadgets, i.e., groups of code statements that have control or dependency relationships with respect to certain interesting code elements such as library/API calls and array usages, to recurrent neural networks for vulnerability detection.
- **VulBERTa** (Hanif & Maffeis, 2022) adopts BERT as strategy, by pre-training a RoBERTa model with MLM (Masked Language Modelling) on a large corpus of C/C++ programs, and then fine-tuning on certain vulnerability datasets to train the vulnerability detectors.
- **Devign** (Zhou et al., 2019) and **Reveal** (Chakraborty et al., 2021) are two graph-based approaches that leverage GGNN (Gated Graph Neural Network) to extract vulnerability indicative features from the CPG (Code Property Graph) (Yamaguchi et al., 2014) representation of the code to be detected.
- Two widely-used models, denoted as Baseline-BiLSTM and Baseline-TextCNN, which take in the lexically parsed token sequence and detect vulnerability with BiLSTM and TextCNN respectively, also serve as baselines.

[5] https://radimrehurek.com/gensim

Table 3
Comparison with SOTA methods on vulnerability presence detection.

Dataset	Method	Acc.	F_1	Prec.	Rec.
Ours	Baseline-BiLSTM	83.12	82.68	81.14	84.28
	Baseline-TextCNN	86.00	85.64	86.49	84.80
	VulDeePecker	87.49	87.38	86.74	88.02
	SySeVR	87.78	87.72	86.67	**88.79**
	Devign	87.21	87.16	86.62	87.50
	TrVD	**89.29**	**88.19**	**92.85**	83.97
D2A	Baseline-BiLSTM	56.42	–	–	–
	Baseline-TextCNN	55.18	–	–	–
	C-BERT	60.20	–	–	–
	Devign	60.34	–	–	–
	VulBERTa	62.30	–	–	–
	TrVD	**64.15**	–	–	–
VulDeePecker	Baseline-BiLSTM	–	66.97	52.58	–
	Baseline-TextCNN	–	75.80	63.48	–
	VulDeePecker	–	92.90	91.90	–
	Devign	–	92.52	93.76	–
	VulBERTa	–	**93.03**	95.76	–
	TrVD	–	92.18	**96.83**	–
Reveal	Baseline-BiLSTM	77.13	39.11	26.76	72.61
	Baseline-TextCNN	73.22	37.42	24.50	**79.13**
	REVEAL	84.37	41.25	30.91	60.91
	Devign	80.71	39.55	28.33	65.47
	VulBERTa	84.48	45.27	35.18	63.48
	TrVD	**86.54**	**47.79**	**38.24**	63.69

Table 3 summarizes the results.[6] As indicated by the values, TrVD outperforms the baseline methods in most of performance metrics across all datasets. To be specific, on our constructed dataset and the Reveal dataset, it achieves the best accuracy, precision and F_1 score against the comparison methods. On the D2 A dataset, TrVD beats other methods with an accuracy of 64.15%, which is 1.85% higher than the second best method VulBERTa. On the VulDeePecker dataset, TrVD wins the third place on F_1 with a score slightly lower than VulDeePecker and VulBERTa, but tops on the precision with a score of 96.83%. This indicates a lower recall for TrVD on this dataset. We can also observe from the right-most column that, the recall scores TrVD achieves on our dataset and the Reveal dataset are basically lower than other methods. The low recall yet impressive precision scores that TrVD achieves imply that TrVD tends to produce more false negatives but much less false positives. But overall, as indicted by the F_1 scores, which are the harmonic mean of precision and recall values, TrVD shows competitive performance among the DL-based methods. Besides, it can be observed that, the scores of all the methods are generally lower on the real-word datasets (i.e., D2 A and Reveal) than on the synthetic/hybrid datasets (i.e., ours and VulDeePecker). This indicates that detecting vulnerability on real-world programs is more challenging, due to more complex and diverse code contexts that vulnerabilities reside in.

> **Answer to RQ1:** Among the DL-based methods evaluated, TrVD shows competitive performances, especially on the accuracy, precision and F_1 metrics. Also, reliable detection on the real-world programs is still challenging, improvements should be made by the DL-based methods to achieve a more satisfactory detection performance.

[6] It should be noted that, not all the comparison methods made public their implementations. We manage to get five baselines evaluated on our dataset, where the results of all the metrics are reported; while on the rest three datasets, we evaluate the performance of TrVD on them and directly compare with the metric values as reported in the existing work (Hanif & Maffeis, 2022).

Table 4
Quantity distribution of vulnerable functions per vulnerability type.

Label	Type	#Sample	Label	Type	#Sample	Label	Type	#Sample	Label	Type	#Sample
1	CWE-22	7600	23	CWE-396	54	45	CWE-672	30	67	CWE-222	18
2	CWE-73	978	24	CWE-398	177	46	CWE-675	300	68	CWE-223	18
3	CWE-77	8200	25	CWE-400	1212	47	CWE-681	5368	69	CWE-247	18
4	CWE-119	27571	26	CWE-404	638	48	CWE-690	1592	70	CWE-273	36
5	CWE-134	6175	27	CWE-426	316	49	CWE-758	581	71	CWE-338	17
6	CWE-176	82	28	CWE-427	805	50	CWE-763	8589	72	CWE-397	6
7	CWE-190	6393	29	CWE-459	108	51	CWE-771	244	73	CWE-475	36
8	CWE-191	4741	30	CWE-464	82	52	CWE-772	2299	74	CWE-478	18
9	CWE-200	36	31	CWE-467	54	53	CWE-789	1516	75	CWE-479	18
10	CWE-252	617	32	CWE-468	37	54	CWE-843	126	76	CWE-483	20
11	CWE-253	684	33	CWE-469	36	55	CWE-908	1039	77	CWE-484	18
12	CWE-271	252	34	CWE-476	484	56	CWE-912	300	78	CWE-535	36
13	CWE-284	216	35	CWE-480	54	57	CWE-943	820	79	CWE-570	16
14	CWE-319	328	36	CWE-534	36	58	CWE-1078	59	80	CWE-571	16
15	CWE-325	72	37	CWE-563	606	59	CWE-1105	36	81	CWE-587	18
16	CWE-327	72	38	CWE-588	121	60	CWE-1177	36	82	CWE-605	18
17	CWE-328	54	39	CWE-606	820	61	CWE-119,672	497	83	CWE-620	18
18	CWE-362	90	40	CWE-617	469	62	CWE-119,672,415	161	84	CWE-785	18
19	CWE-369	1387	41	CWE-628	36	63	CWE-1390,344	319	85	CWE-835	6
20	CWE-377	144	42	CWE-665	311	64	CWE-1390,522	64			
21	CWE-390	90	43	CWE-666	90	65	CWE-15,642	82			
22	CWE-391	54	44	CWE-667	200	66	CWE-459,212	72			

Table 5
Comparison with SOTA methods on vulnerability type detection.

Dataset	Method	Acc	Weighted F_1	Weighted Prec	Weighted Rec
Ours	Baseline-BiLSTM	74.54	68.38	64.55	74.53
	Baseline-TextCNN	76.21	70.81	70.05	76.21
	VulDeePecker	83.31	81.93	81.86	83.31
	SySeVR	84.88	84.21	84.86	84.89
	Devign	83.47	82.49	84.27	83.47
	TrVD	**85.85**	**85.54**	**85.98**	**85.83**
muVulDeePecker	Baseline-BiLSTM	–	65.93	–	–
	Baseline-TextCNN	–	65.95	–	–
	VulDeePecker	–	93.61	–	–
	muVulDeePecker	–	94.69	–	–
	VulBERTa	–	**99.59**	–	–
	TrVD	–	94.93	–	–

5.2.2. RQ2: Performance on vulnerability type detection

In this experiment, we evaluate the performance of TrVD in pinpointing the specific vulnerability types. Our constructed dataset, which consists of 98,181 vulnerable functions that are well-labeled to 118 different CWE-IDs, is used for the evaluation, considering the adequacy of the vulnerable samples and the richness of the vulnerability types it contains. As the CWE-IDs are organized hierarchically (i.e., low-level CWE-IDs are affiliated to some higher-level CWE-IDs, e.g., CWE-119 includes CWE-787 and CWE-788 as sub-types), we adopt the same policy as in muVulDeePecker (Zou et al., 2021) to aggregate the CWE-IDs to the third level of the CWE-ID tree in its research concept view, and regard these third-level CWE-IDs as the vulnerability types. For example, for a vulnerable sample originally marked as CWE-787 or the lower-level type CWE-121, we reassign its vulnerability type to the CWE-ID at the third level of the CWE-ID tree, which is CWE-119 in this case. Also, there are CWE-IDs that are children of multiple third-level CWE-IDs. For example, the low-level CWE-416 can be attributed to either the CWE-119 or CWE-672 at the third level of the CWE-ID tree. For the presentation consistency, we use their joint (i.e., "CWE-119, CWE-672") to denote the vulnerability type for CWE-416. Finally, this leads to 85 different types as listed in Table 4.

It should be noted that, mulVulDeePecker (Zou et al., 2021) also publishes its dataset for vulnerability type detection.[7] However, as the source code of mulVulDeePecker is not made public, we only evaluate and compare the performance of TrVD on its dataset, which involves 40 vulnerability types.

Table 5 summarizes the experimental results. It can be observed that, TrVD surpasses the comparison methods on all the performance metrics on our dataset, while underperforms VulBERTa and muVulDeePecker on the muVulDeePecker dataset, with a weighted F_1 score of 94.93%. We argue that it is still an acceptable score, considering our original aim of building TrVD as a detection method that prioritizes accuracy, efficiency and practical applicability simultaneously.

> **Answer to RQ2:** TrVD is effective for the multi-class vulnerability type detection task. It shows promising and competitive performance among the DL-based baseline methods.

5.2.3. RQ3: Ablation study on TrVD's core components

As illustrated by the evaluation results in the above sections, as a whole, TrVD outperforms SOTA methods in detecting either the presence or the specific types of the given vulnerabilities. We attribute such superior detection performance to the elaborate designs in TrVD's core components. Hereby, we conduct the ablation study as follows to better understand the roles that TrVD's different components play in contributing to the overall detection performance.

(1) The effects of the AST Decomposer

As discussed in Sections 2.3 and 4.2, we design the decomposer to convert a full AST into a sequence of sub-trees of restricted sizes, with the purpose of alleviating the cost of subsequent neural encoders in handling large/deep ASTs, and better disclosing the subtle vulnerability semantics conveyed in the small-sized code unit. To evaluate its effects, we substitute our decomposer with the following alternatives and compare their detection performances on both vulnerability presence detection and vulnerability type detection tasks. Specifically, these alternatives include:

- AST_{TBCNN}, where neither decomposition nor serialization is enforced, but the whole AST is directly fed into the tree-based convolutional neural network (TBCNN) (Mou et al., 2016) for feature extraction;
- $AST_{TreeLSTM}$, where the whole AST is directly fed into the tree-structured long short-term memory network (Tree-LSTM) (Tai et al., 2015) for feature extraction;

[7] For this task, we did not use other datasets, due to their unavailability of the mappings between the vulnerable samples and the vulnerability types.

Table 6
Effects of the AST Decomposer.

(a) Performance on Vulnerability Presence Detection

Method	Dataset	Acc	F_1	Prec	Rec	Dataset	Acc	F_1	Prec	Rec
AST_{TBCNN}	Ours	87.71	87.62	86.88	**88.37**	VulDeePecker	90.01	91.08	94.08	88.26
$AST_{TreeLSTM}$		86.81	86.55	86.78	86.33		90.23	91.88	94.59	**89.33**
$AST_{AttRvNN}$		87.36	87.25	86.83	87.61		91.68	91.40	94.62	88.40
AST_{RvNN}		87.01	86.75	87.03	86.48		91.17	90.92	93.57	88.40
TrVD		**89.29**	**88.19**	**92.85**	83.97		**92.52**	**92.18**	**96.83**	87.95
AST_{TBCNN}	D2A	58.89	61.90	59.40	64.61	Reveal	83.45	42.53	30.88	68.28
$AST_{TreeLSTM}$		58.72	60.03	58.11	62.08		83.19	42.24	30.52	**68.57**
$AST_{AttRvNN}$		59.08	62.21	58.95	**65.86**		83.98	43.05	32.17	65.03
AST_{RvNN}		58.55	61.16	58.73	63.79		83.66	42.59	31.30	66.61
TrVD		**64.15**	**63.14**	**67.34**	59.43		**86.54**	**47.79**	**38.24**	63.69

(b) Performance on Vulnerability Type Detection

Method	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec
AST_{TBCNN}	Ours	82.54	84.78	85.27	84.55	mu-VulDeePecker	89.42	88.72	88.94	89.42
$AST_{TreeLSTM}$		81.89	83.64	84.47	83.90		88.11	87.44	87.47	88.11
$AST_{AttRvNN}$		83.94	84.64	85.05	83.90		91.95	91.61	91.62	91.95
AST_{RvNN}		82.47	84.55	85.55	84.47		90.74	90.23	90.40	90.74
TrVD		**85.85**	**85.54**	**85.98**	**85.83**		**95.01**	**94.93**	**95.12**	**95.01**

- ASTNN (Zhang et al., 2019), where the AST is also decomposed into a set of sub-trees, but differently, compound statements are always split as described in Section 4.2.

Table 6 summarizes the experimental results. As indicated by the values in terms of the performance metrics, TrVD holds a lead against its alternatives with better accuracy, precision and F_1 scores for both tasks. The performance gains raised by our Decomposer are obvious, with averagely 4.23% improvement on accuracy, 4.36% improvement on F_1 and 11.68% improvement on precision scores on the vulnerability presence detection task, and averagely 4.67% improvement on accuracy, 3.73% improvement on F_1 and 3.68% improvement on precision scores on the vulnerability type detection task. This demonstrates the effectiveness of our tree decomposition algorithm, which allows the subtle semantics implied deep in the code to be better captured by the subsequent neural encoders. Meantime, we notice that the comparison methods slightly outperform TrVD in terms of the recall scores on the vulnerability presence detection task. Considering the impressive gains on the precision metric, the introduction of tree decomposition enforces TrVD to deliver more false negatives while avoid many more false positives. But again, the higher F_1 score indicates an overall positive effect of the Decomposer.

(2) The effects of the Semantics Collector

As discussed in Section 4.3, we design an attention-augmented tree-structured RvNN to collect the semantics from a sub-tree. To evaluate its effects, we substitute it with other well-adopted tree-oriented neural networks and compare their performances, including:

- $TrVD_{TBCNN}$, where the TBCNN is used to encode each sub-tree;
- $TrVD_{TreeLSTM}$, where the Tree-LSTM is used to encode each sub-tree;
- $TrVD_{RvNN}$, where the naive RvNN without attention mechanism is used to encode each sub-tree.

Table 7 reports the experimental results. As the values show, TrVD basically performs better than its alternatives, validating the superiority of our attention-augmented RvNN encoder against other tree-oriented neural encoders in collecting the latent code semantics. In addition, these less significant performance improvements than the ones observed in the last experiment implies that, the Decomposer plays a more impactful role than the attention-augmented RvNN semantic Collector in contributing to the competitive detection performance of TrVD.

(3) The effects of the Semantics Focuser

In Section 4.4, we resort to a Transformer-style neural network to aggregate the semantics collected from the decomposed sub-trees into a dense vulnerability-specific vector. To evaluate its effects, we substitute it with other aggregation strategies and compare their performances in this set of experiments. The specific replacement strategies include $TrVD_{AvgPooling}$, $TrVD_{MaxPooling}$, $TrVD_{TextCNN}$, $TrVD_{DPCNN}$ and $TrVD_{BiLSTM}$, where the average pooling, max pooling, TextCNN (Kim, 2014), DPCNN (Johnson & Zhang, 2017) and BiLSTM are used respectively to aggregate the semantic vectors collected from all the sub-trees of an AST.

Table 8 summarizes the comparative results. A similar observation can be made that TrVD outperforms its alternatives in most cases, which demonstrates the superiority of adopting a Transformer-style neural network as the aggregator to better learn contextual long-range dependencies among the sub-trees and enable the more informative semantics that are indicative of the vulnerability patterns to be attended. Again, despite the performance improvements that are less impressive than the Decomposer, our design in the Focuser does bring positive effects.

> **Answer to RQ3:** The core designs in TrVD's Decomposer, Collector and Focuser all play active roles, with the well-designed tree decomposition algorithm in the Decomposer contributing the most to the competitive detection performance of TrVD.

5.2.4. RQ4: Efficiency analysis

In this section, we analyze the runtime overhead of TrVD. Given a code snippet, it takes TrVD two major steps, the data preparation step and the neural classification step, to accomplish its detection. The data preparation step, supported by the *AST Generator* and *AST Decomposer* in TrVD, extracts the AST from the input code snippet and decomposes it into a sequence of sub-trees. The neural classification step, supported by the *Semantics Collector*, *Suspicious Semantics Focuser* and *Vulnerability Detector* in TrVD, makes a prediction with the trained neural network model.

Specifically, we feed each sample from our dataset[8] into TrVD for detection, and meanwhile record the time consumed in the two steps

[8] When comparing TrVD to other baseline approaches, its overhead is assessed solely using the samples from our curated dataset, where we manage to operationalize five baseline methods. Furthermore, we exclusively report the runtime overheads associated with vulnerability presence detection. Notably, the runtime overheads for vulnerability type detection are similar, as only the final softmax classification operation is different.

Table 7
Effects of the Semantics Collector.

(a) Performance on Vulnerability Presence Detection										
Method	Dataset	Acc	F_1	Prec	Rec	Dataset	Acc	F_1	Prec	Rec
$TrVD_{TBCNN}$	Ours	88.73	84.87	85.68	**84.07**	VulDeePecker	92.31	92.15	94.39	89.98
$TrVD_{TreeLSTM}$		89.09	85.13	86.38	83.91		92.01	91.76	94.83	**88.88**
$TrVD_{RvNN}$		89.11	85.35	88.22	82.66		92.41	92.12	96.09	88.47
TrVD		**89.29**	**88.19**	**92.85**	83.97		**92.52**	**92.18**	**96.83**	87.95
$TrVD_{TBCNN}$	D2A	62.45	61.99	62.46	**61.53**	Reveal	85.81	46.66	36.48	64.72
$TrVD_{TreeLSTM}$		63.05	62.17	65.30	59.33		86.32	47.30	37.53	**67.93**
$TrVD_{RvNN}$		62.42	61.78	65.11	58.77		86.03	46.69	36.76	63.98
TrVD		**64.15**	**63.14**	**67.34**	59.43		**86.54**	**47.79**	**38.24**	63.69
(b) Performance on Vulnerability Type Detection										
Method	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec
$TrVD_{TBCNN}$	Ours	85.39	84.94	85.38	85.39	mu-VulDeePecker	93.55	94.18	94.37	94.25
$TrVD_{TreeLSTM}$		84.71	85.34	85.93	85.71		94.25	94.90	95.04	94.98
$TrVD_{RvNN}$		85.67	85.32	85.78	85.68		94.98	93.43	93.84	93.55
TrVD		**85.85**	**85.54**	**85.98**	**85.83**		**95.01**	**94.93**	**95.12**	**95.01**

Table 8
Effects of the Semantics Focuser.

(a) Performance on Vulnerability Presence Detection										
Method	Dataset	Acc	F_1	Prec	Rec	Dataset	Acc	F_1	Prec	Rec
$TrVD_{AvgPooling}$	Ours	88.31	87.66	86.97	**88.37**	VulDeePecker	91.74	91.48	94.40	**88.74**
$TrVD_{MaxPooling}$		88.86	87.62	90.85	84.61		91.63	91.32	94.88	88.02
$TrVD_{TextCNN}$		88.46	86.98	90.63	83.62		91.84	91.57	94.70	88.64
$TrVD_{DPCNN}$		89.12	87.33	**92.87**	82.42		91.79	91.50	94.80	88.43
$TrVD_{BiLSTM}$		88.73	87.07	88.83	85.37		91.93	91.69	94.94	88.65
TrVD		**89.29**	**88.19**	92.85	83.97		**92.52**	**92.18**	**96.83**	87.95
$TrVD_{AvgPooling}$	D2A	61.96	62.76	61.09	**64.53**	Reveal	85.66	46.93	36.82	64.71
$TrVD_{MaxPooling}$		62.50	62.54	61.92	63.17		85.82	46.97	36.80	**64.92**
$TrVD_{TextCNN}$		62.32	61.83	62.34	61.32		85.97	47.01	37.06	64.27
$TrVD_{DPCNN}$		63.49	62.16	64.76	59.76		86.29	47.37	37.54	64.18
$TrVD_{BiLSTM}$		63.23	61.61	63.82	60.15		86.12	47.31	37.44	64.25
TrVD		**64.15**	**63.14**	**67.34**	59.43		**86.54**	**47.79**	**38.24**	63.69
(b) Performance on Vulnerability Type Detection										
Method	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec	Dataset	Acc	Weighted F_1	Weighted Prec	Weighted Rec
$TrVD_{AvgPooling}$	Ours	73.34	66.74	62.53	73.34	mu-VulDeePecker	92.06	91.70	91.73	92.06
$TrVD_{MaxPooling}$		75.71	70.01	67.79	75.71		92.35	92.11	92.40	92.35
$TrVD_{TextCNN}$		84.41	83.57	83.99	84.41		94.72	94.62	94.73	94.72
$TrVD_{DPCNN}$		84.88	84.20	84.86	84.88		93.17	93.11	93.36	93.17
$TrVD_{BiLSTM}$		85.06	84.59	85.26	85.06		94.96	94.87	95.03	94.96
TrVD		**85.85**	**85.54**	**85.98**	**85.83**		**95.01**	**94.93**	**95.12**	**95.01**

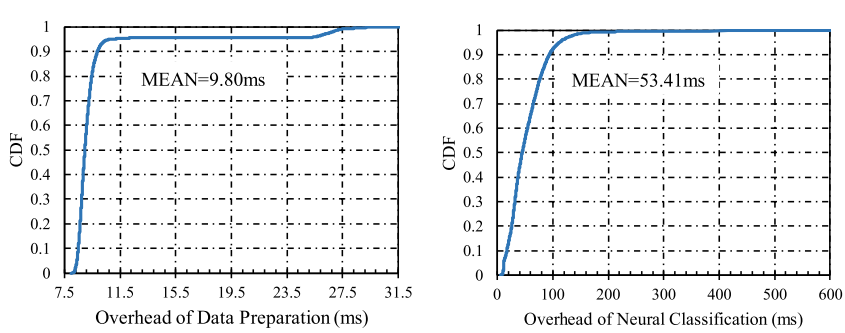

Fig. 4. Evaluation on the runtime overhead of TrVD's two major steps.

respectively. As depicted in Fig. 4 about the runtime overheads, TrVD consumes averagely 9.8 ms in obtaining the sequence of decomposed sub-trees, and 53.41 ms in making a prediction with the trained model.

To give a more intuitive sense of the efficiency of TrVD, we also compare its runtime overhead with five other DL-based methods, including Baseline-BiLSTM, Baseline-TextCNN, VulDeePecker, SySeVR and Devign. Their average runtime overheads are reported in Fig. 5.

As it shows, Baseline-TextCNN and Baseline-BiLSTM are the most efficient methods, since they only apply simple lexical analysis to obtain the source code tokens and use a CNN and BiLSTM model to detect vulnerability respectively. On average, it takes them only 25 ms and 38 ms respectively to finish the analysis of a function in our dataset. The graph-based method Devign is the most computationally intensive, due to its heavy weight code property graph construction phase and

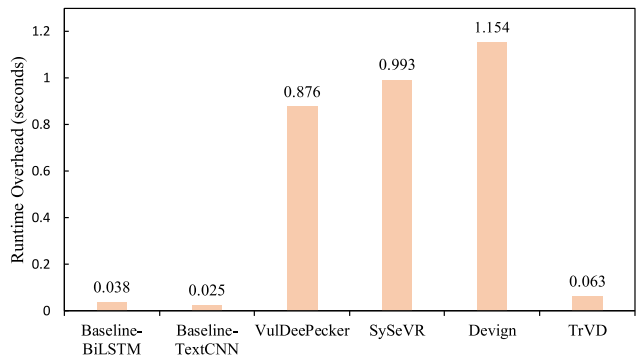

Fig. 5. Runtime comparison with other methods.

the GGNN based feature extraction phase. Averagely, it takes 1154 ms to analyze a function, which is about 46 times slower than Baseline-TextCNN. As for VulDeePecker and SySeVR, code slicing using control or data dependency is enforced to obtain code gadgets from the PDG, which makes their data preparation phase heavy-weight. Accordingly, VulDeePecker and SySeVR take averagely 876 ms and 993 ms to detect a function, while TrVD takes 63 ms on average to analyze a function, which, though not as efficient as the token-based methods, is about 14–16 times faster than VulDeePecker and SySeVR.

Answer to RQ4: By following a light-weight data preparation strategy, TrVD is significantly learning-effective yet cost-efficient among the DL-based baseline methods.

6. Discussion

6.1. Applicability issues

As discussed in Section 2.2, TrVD opts to operate on the easy-to-obtain AST to ensure its applicability. Other purposely-crafted code representations, such as PDG and code gadgets, are difficult to precisely construct for non-compilable code fragments or simply cannot be extracted without the presence of certain code elements in the code (thus severely diminish the applicability of the detection methods that rely on these representations). By contrast, the AST can always be statically constructed for almost any code fragment. With the tree-sitter parsing library, TrVD succeeds in analyzing each single sample from all five datasets we evaluated on. Taking our constructed dataset as an example, TrVD is able to process all the functions, while the baseline methods SySeVR and VulDeePecker only allow 60.5% of the whole samples to be successfully analyzed. Also, both our dataset and VulDeePeecker's dataset are mainly constructed from the SARD dataset, but the number of samples in their dataset (i.e., the ones that can be processed by their method) is far less, which is 68.6% of ours.

In this work, TrVD is evaluated on the code written in C and C++, which are among the programming languages that have been hit hard by vulnerabilities. But theoretically, it can be extended to the vulnerability detection for code written in other programming languages in a straightforward way, as long as the AST can be constructed from the target code snippets. The tree-sitter library that TrVD relies upon to construct ASTs, currently provides fairly mature parsing ability for over 40 different kinds of programming languages. In our future work, we plan to extend TrVD to support the vulnerability detection for more programming languages, such as the widely-used Java and the emerging Solidity used for smart contract development. Definitely, the tree decomposition algorithm as well as the neural encoding and classification models can be adjusted to accommodate to the specific characteristics of the language to be analyzed, so as to ensure an optimal detection performance.

As a learning-based detection method, TrVD still suffers from the limitation of only reporting black-box detection results, i.e., a vulnerable/non-vulnerable prediction or the specific vulnerability type without explaining, which is different from the rule-based methods by providing detection results with additional information to imply possible bug-trigging paths of the detected vulnerabilities (Cheng, Nie et al., 2022). Driven by this purpose, several works have emerged recently with the goal of better explaining the detection results, by highlighting the statements (Ding et al., 2022; Li, Wang et al., 2021; Li, Zou, Xu, Chen et al., 2022; Nguyen et al., 2021) or paths (Cheng, Zhang et al., 2022) with significant contributions to the prediction outcomes using either explainable AI techniques, such as GNN-explainer (Li, Wang et al., 2021) and mutual information maximization (Nguyen et al., 2021), or jointly-trained multi-grained models (Cheng, Zhang et al., 2022; Ding et al., 2022). TrVD can be also adjusted to highlight the indicative sub-trees (i.e., statements) with the attention scores or other explainable AI techniques (e.g., local approximation methods). We leave this as an interesting future work.

6.2. Dataset issues

As pointed out by existing works (Cheng, Zhang et al., 2022; Croft, Babar, & Kholoosi, 2023; Croft, Xie, & Babar, 2023; Hanif & Maffeis, 2022; Jimenez et al., 2019; Tian, Tian, Lv, & Chen, 2023), the sample mislabeling in the vulnerability datasets remains a common and unresolved problem. But the mislabeling ratio in the datasets we used is relatively low, as the included samples are labeled either by security experts or through a well-designed differential analysis technique (Zheng et al., 2021). Thus, as a DL-based method, TrVD should be resilient to the occasional label noises during model training. The influence of the low-ratio noises on the testing performance should also be minor.

Those real-world vulnerability datasets with satisfying annotations are critically lacking. The datasets we used are primarily labeled at the same granularity of isolated functions as the one used by the majority of existing works. In practice, vulnerabilities can happen across function boundaries. It is thus not always correct to mark a function as vulnerable simply because the vulnerability is disclosed in this function, while additional inter-procedural information, such as the macros and the functions that it calls, should be also taken into consideration to form the complete bug-trigging contexts. Unfortunately, constructing such in-scale datasets that are comprised of precisely-labeled bug-trigging code contexts is non-trivial, which requires continuous laborious efforts from domain experts. But even so, we argue that TrVD should work on such data with moderate adjustments, since all its components are not restricted to be applied on isolated functions.

The numbers of samples that reside in different classes are imbalanced in some of our datasets. For example, the ratio between the vulnerable and non-vulnerable samples in the Reveal dataset is about 1:10. During the model training, we under-sample the non-vulnerable samples to obtain balanced training sets. Other data imbalance handling techniques, such as SMOTE re-sampling, may also be feasible. Besides, the emerging bug and vulnerability injection techniques (Ding et al., 2022; Nong, Ou, Pradel, Chen, & Cai, 2022; Yu, Yuzhe, Michael, Feng, & Haipeng, 2023) deserve further investigation and adaptation to enrich the diversity and the size of vulnerable samples, which help solve the data-imbalance issue and may improve the detection performance of DL-based methods. We leave them as future works.

7. Conclusion

In this work, we present TrVD, a novel source code vulnerability detection approach that follows the DL-based paradigm. Through the comprehensive contributions from our elaborate designs, including the novel tree decomposition algorithm, the attention-augmented sub-tree

encoder and the vulnerability-indicative semantic focuser, TrVD turns out to be accurate, efficient and practically applicable in detecting either the presence of vulnerability or the specific vulnerability type. The extensive experiments conducted on five large datasets consisting of both synthetic and real-word samples demonstrate the superiority of TrVD against SOTA DL-based methods in the aspects of detection performance and runtime overhead. The effectiveness of TrVD's core designs are also well-studied with a thorough ablation study. As our future works, we plan to extend TrVD to investigate advanced embedding models (such as large-scale pre-trained models, e.g., CodeBert) for token representation learning, more diverse programming languages, and interpretations for the detection results.

CRediT authorship contribution statement

Zhenzhou Tian: Conceptualization, Methodology, Supervision. **Binhui Tian:** Investigation, Writing – original draft, Software. **Jiajun Lv:** Data curation, Validation. **Yanping Chen:** Writing – review & editing. **Lingwei Chen:** Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Zhenzhou Tian reports financial support was provided by National Natural Science Foundation of China. Zhenzhou Tian reports financial support was provided by Natural Science Basic Research Program of Shaanxi Province. Zhenzhou Tian reports financial support was provided by the Youth Innovation Team of Shaanxi Universities.

Data availability

I have shared the link to my data/code in the manuscript.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62272387, 61702414), the Natural Science Basic Research Program of Shaanxi, China (2022JM-342, 2018JQ6078), the Youth Innovation Team of Shaanxi Universities "Industrial Big Data Analysis and Intelligent Processing", China, and the Special Funds for Construction of Key Disciplines in Universities in Shaanxi, China.

References

A. Wheeler, D. (2014). A static analysis tool for finding vulnerabilities in c/c++ source code. https://github.com/david-a-wheeler/flawfinder.

Alon, U., Levy, O., & Yahav, E. (2019). Code2seq: Generating sequences from structured representations of code. In *International conference on learning representations*.

Alon, U., Zilberstein, M., Levy, O., & Yahav, E. (2019). Code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, *3*(POPL), http://dx.doi.org/10.1145/3290353.

Bosu, A., Carver, J. C., Hafiz, M., Hilley, P., & Janni, D. (2014). Identifying the characteristics of vulnerable code changes: An empirical study. In *FSE 2014, Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering* (pp. 257–268). New York, NY, USA: Association for Computing Machinery.

Bui, N. D. Q., Yu, Y., & Jiang, L. (2021). InferCode: Self-supervised learning of code representations by predicting subtrees. In *2021 IEEE/ACM 43rd international conference on software engineering* (pp. 1186–1197). http://dx.doi.org/10.1109/ICSE43902.2021.00109.

Cao, S., Sun, X., Bo, L., Wei, Y., & Li, B. (2021). BGNN4vd: Constructing bidirectional graph neural-network for vulnerability detection. *Information and Software Technology*, *136*, Article 106576.

Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2021). Deep learning based vulnerability detection: Are we there yet. *IEEE Transactions on Software Engineering*, 1. http://dx.doi.org/10.1109/TSE.2021.3087402.

Checkmarx (2022). Checkmarx. https://www.checkmarx.com.

Cheng, X., Nie, X., Li, N., Wang, H., Zheng, Z., & Sui, Y. (2022). How about bug-triggering paths? - understanding and characterizing learning-based vulnerability detectors. *IEEE Transactions on Dependable and Secure Computing*, 1–18. http://dx.doi.org/10.1109/TDSC.2022.3192419.

Cheng, X., Wang, H., Hua, J., Xu, G., & Sui, Y. (2021). DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology*, *30*(3), http://dx.doi.org/10.1145/3436877.

Cheng, X., Wang, H., Hua, J., Zhang, M., Xu, G., Yi, L., et al. (2019). Static detection of control-flow-related vulnerabilities using graph embedding. In *2019 24th International conference on engineering of complex computer systems* (pp. 41–50).

Cheng, X., Zhang, G., Wang, H., & Sui, Y. (2022). Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *ISSTA 2022, Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis* (pp. 519–531). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3533767.3534371.

CodeChecker (2013). CodeChecker: A static analysis infrastructure built on the LLVM/Clang static analyzer. https://codechecker.readthedocs.io/en/latest.

Croft, R., Babar, M. A., & Kholoosi, M. (2023). Data quality for software vulnerability datasets. In *Proceedings of the 2023 IEEE/ACM international conference on software engineering* (p. 1).

Croft, R., Xie, Y., & Babar, M. (2023). Data preparation for software vulnerability prediction: A systematic literature review. *IEEE Transactions on Software Engineering*, *49*(03), 1044–1063. http://dx.doi.org/10.1109/TSE.2022.3171202.

Cui, L., Cui, J., Hao, Z., Li, L., Ding, Z., & Liu, Y. (2022). An empirical study of vulnerability discovery methods over the past ten years. *Computers & Security*, *120*(C), http://dx.doi.org/10.1016/j.cose.2022.102817.

Cui, L., Hao, Z., Jiao, Y., Fei, H., & Yun, X. (2021). VulDetector: Detecting vulnerabilities using weighted feature graph comparison. *IEEE Transactions on Information Forensics and Security*, *16*, 2004–2017. http://dx.doi.org/10.1109/TIFS.2020.3047756.

CVE (2023). CVE. https://cve.mitre.org.

CWE (2023). Common weakness enumeration. https://cwe.mitre.org/.

Ding, Y., Suneja, S., Zheng, Y., Laredo, J., Morari, A., Kaiser, G., et al. (2022). VELVET: A novel ensemble learning approach to automatically locate VulnErable statements. In *2022 IEEE international conference on software analysis, evolution and reengineering* (pp. 959–970). http://dx.doi.org/10.1109/SANER53432.2022.00114.

Ghaffarian, S. M., & Shahriari, H. R. (2017). Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. *ACM Computing Surveys*, *50*(4), http://dx.doi.org/10.1145/3092566.

Google (2013). Rough audit tool for security. https://code.google.com/archive/p/rough-auditing-tool-for-security/.

Goseva-Popstojanova, K., & Perhinschi, A. (2015). On the capability of static code analysis to detect security vulnerabilities. *Information and Software Technology*, *68*, 18–33. http://dx.doi.org/10.1016/j.infsof.2015.08.002.

Grieco, G., Grinblat, G. L., Uzal, L., Rawat, S., Feist, J., & Mounier, L. (2016). Toward large-scale vulnerability discovery using machine learning. In *Proceedings of the sixth ACM conference on data and application security and privacy* (pp. 85–96). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2857705.2857720.

Hanif, H., & Maffeis, S. (2022). Vulberta: Simplified source code pre-training for vulnerability detection. In *2022 International joint conference on neural networks* (pp. 1–8). http://dx.doi.org/10.1109/IJCNN55064.2022.9892280.

Harrold, M. J., Malloy, B., & Rothermel, G. (1993). Efficient construction of program dependence graphs. In *Proceedings of the 1993 ACM SIGSOFT international symposium on software testing and analysis* (pp. 160–170). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/154183.154268.

Hou, S., Chen, L., & Ye, Y. (2022). Summarizing source code from structure and context. In *2022 International joint conference on neural networks* (pp. 1–8). IEEE.

Hu, X., Li, G., Xia, X., Lo, D., & Jin, Z. (2018). Deep code comment generation. In *2018 IEEE/ACM 26th international conference on program comprehension* (pp. 200–20010).

Infer (2013). Infer: A tool to detect bugs in java and c/c++/objective-c code before it ships. https://fbinfer.com.

Jang, J., Agrawal, A., & Brumley, D. (2012). ReDeBug: Finding unpatched code clones in entire OS distributions. In *IEEE symposium on security and privacy, SP 2012, 21-23 May 2012, San Francisco, California, USA* (pp. 48–62). IEEE Computer Society, http://dx.doi.org/10.1109/SP.2012.13.

Jiang, X., Zheng, Z., Lyu, C., Li, L., & Lyu, L. (2021). Treebert: A tree-based pre-trained model for programming language. In C. de Campos, & M. H. Maathuis (Eds.), *Proceedings of machine learning research: vol. 161, Proceedings of the thirty-seventh conference on uncertainty in artificial intelligence* (pp. 54–63). PMLR.

Jimenez, M., Rwemalika, R., Papadakis, M., Sarro, F., Le Traon, Y., & Harman, M. (2019). The importance of accounting for real-world labelling when predicting software vulnerabilities. In *ESEC/FSE 2019, Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering* (pp. 695–705). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3338906.3338941.

Johnson, R., & Zhang, T. (2017). Deep pyramid convolutional neural networks for text categorization. In *Proceedings of the 55th annual meeting of the association for computational linguistics (Volume 1: Long papers)* (pp. 562–570). Vancouver, Canada: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/P17-1052.

Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017). Bag of tricks for efficient text classification. In *Proceedings of the 15th conference of the european chapter of the association for computational linguistics: Volume 2, Short papers* (pp. 427–431). Valencia, Spain: Association for Computational Linguistics.

Kang, W., Son, B., & Heo, K. (2022). TRACER: Signature-based static analysis for detecting recurring vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security* (pp. 1695–1708). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3548606.3560664.

Kim, Y. (2014). Convolutional neural networks for sentence classification. In *Proceedings of the 2014 conference on empirical methods in natural language processing* (pp. 1746–1751). Doha, Qatar: Association for Computational Linguistics, http://dx.doi.org/10.3115/v1/D14-1181.

Kim, S., Woo, S., Lee, H., & Oh, H. (2017a). VUDDY: A scalable approach for vulnerable code clone discovery. In *2017 IEEE symposium on security and privacy* (pp. 595–614). http://dx.doi.org/10.1109/SP.2017.62.

Kim, S., Woo, S., Lee, H., & Oh, H. (2017b). VUDDY: A scalable approach for vulnerable code clone discovery. In *2017 IEEE symposium on security and privacy* (pp. 595–614). http://dx.doi.org/10.1109/SP.2017.62.

Kim, S., Zhao, J., Tian, Y., & Chandra, S. (2021). Code prediction by feeding trees to transformers. In *2021 IEEE/ACM 43rd international conference on software engineering* (pp. 150–162). http://dx.doi.org/10.1109/ICSE43902.2021.00026.

Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

Li, X., Chen, L., & Wu, D. (2021). Turning attacks into protection: Social media privacy protection using adversarial attacks. In *Proceedings of the 2021 SIAM international conference on data mining* (pp. 208–216). SIAM.

Li, Z., Lu, S., Myagmar, S., & Zhou, Y. (2006). CP-miner: finding copy-paste and related bugs in large-scale software code. *IEEE Transactions on Software Engineering*, *32*(3), 176–192. http://dx.doi.org/10.1109/TSE.2006.28.

Li, Y., Wang, S., & Nguyen, T. N. (2021). Vulnerability detection with fine-grained interpretations. In *ESEC/FSE 2021, Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering* (pp. 292–303). New York, NY, USA: Association for Computing Machinery.

Li, Y., Wang, S., Nguyen, T. N., & Van Nguyen, S. (2019). Improving bug detection via context-based code representation learning and attention-based neural networks. *Proceedings of the ACM on Programming Languages*, *3*(OOPSLA), http://dx.doi.org/10.1145/3360588.

Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., & Jin, H. (2022). VulDeeLocator: A deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing*, *19*(4), 2821–2837. http://dx.doi.org/10.1109/TDSC.2021.3076142.

Li, Z., Zou, D., Xu, S., Jin, H., Qi, H., & Hu, J. (2016). VulPecker: An automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd annual conference on computer security applications* (pp. 201–213). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2991079.2991102.

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., & Chen, Z. (2022). SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, *19*(4), 2244–2258. http://dx.doi.org/10.1109/TDSC.2021.3051525.

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., et al. (2018). VulDeePecker: A deep learning-based system for vulnerability detection. In *25th Annual network and distributed system security symposium*. The Internet Society.

Lin, G., Wen, S., Han, Q.-L., Zhang, J., & Xiang, Y. (2020). Software vulnerability detection using deep neural networks: A survey. *Proceedings of the IEEE*, *108*(10), 1825–1848. http://dx.doi.org/10.1109/JPROC.2020.2993293.

Lin, G., Zhang, J., Luo, W., Pan, L., & Xiang, Y. (2017). POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security* (pp. 2539–2541). New York, NY, USA: Association for Computing Machinery.

Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., De Vel, O., et al. (2018). Cross-project transfer representation learning for vulnerable function discovery. *IEEE Transactions on Industrial Informatics*, *14*(7), 3289–3297. http://dx.doi.org/10.1109/TII.2018.2821768.

Lipp, S., Banescu, S., & Pretschner, A. (2022). An empirical study on the effectiveness of static c code analyzers for vulnerability detection. In *ISSTA 2022*, (pp. 544–555). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3533767.3534380.

Marjamaki, D. (2013). Cppcheck. https://github.com/danmar/cppcheck.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. In *1st International conference on learning representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop track proceedings*.

Moore, M. (2022). 2022 State of open source security report. https://snyk.io/blog/announcing-2022-state-of-open-source-security-report.

Morrison, P., Herzig, K., Murphy, B., & Williams, L. (2015). Challenges with applying vulnerability prediction models. In *Proceedings of the 2015 symposium and bootcamp on the science of security*. New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2746194.2746198.

Mou, L., Li, G., Zhang, L., Wang, T., & Jin, Z. (2016). Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the thirtieth AAAI conference on artificial intelligence* (pp. 1287–1293). AAAI Press.

Nguyen, V., Le, T., De Vel, O., Montague, P., Grundy, J., & Phung, D. (2021). Information-theoretic source code vulnerability highlighting. In *2021 International joint conference on neural networks* (pp. 1–8). http://dx.doi.org/10.1109/IJCNN52387.2021.9533907.

Niu, C., Li, C., Ng, V., Ge, J., Huang, L., & Luo, B. (2022). SPT-code: Sequence-to-sequence pre-training for learning source code representations. In *2022 IEEE/ACM 44th international conference on software engineering* (pp. 01–13). http://dx.doi.org/10.1145/3510003.3510096.

Nong, Y., Ou, Y., Pradel, M., Chen, F., & Cai, H. (2022). Generating realistic vulnerabilities via neural code editing: An empirical study. In *ESEC/FSE 2022, Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering* (pp. 1097–1109). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3540250.3549128.

Pennington, J., Socher, R., & Manning, C. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing* (pp. 1532–1543). Doha, Qatar: Association for Computational Linguistics, http://dx.doi.org/10.3115/v1/D14-1162.

Perl, H., Dechand, S., Smith, M., Arp, D., Yamaguchi, F., Rieck, K., et al. (2015). VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security* (pp. 426–437). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2810103.2813604.

Pham, N. H., Nguyen, T. T., Nguyen, H. A., & Nguyen, T. N. (2010). Detection of recurring software vulnerabilities. In *Proceedings of the IEEE/ACM international conference on automated software engineering* (pp. 447–456). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/1858996.1859089.

Pradel, M., & Sen, K. (2018). *DeepBugs: A learning approach to name-based bug detection, Vol. 2*. (OOPSLA), New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3276517.

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., et al. (2018). Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications* (pp. 757–762). http://dx.doi.org/10.1109/ICMLA.2018.00120.

Shi, M., Tang, Y., Zhu, X., Zhuang, Y., Lin, M., & Liu, J. (2022). Feature-attention graph convolutional networks for noise resilient learning. *IEEE Transactions on Cybernetics*, *52*(8), 7719–7731. http://dx.doi.org/10.1109/TCYB.2022.3143798.

Sun, H., Cui, L., Li, L., Ding, Z., Hao, Z., Cui, J., et al. (2021). Vdsimilar: Vulnerability detection based on code similarity of vulnerabilities and patches. *Computers & Security*, *110*, Article 102417. http://dx.doi.org/10.1016/j.cose.2021.102417.

Svyatkovskiy, A., Zhao, Y., Fu, S., & Sundaresan, N. (2019). Pythia: AI-assisted code completion system. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery* (pp. 2727–2735). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3292500.3330699.

Synopsys (2022). Coverity scan static analysis. https://scan.coverity.com/.

Tai, K. S., Socher, R., & Manning, C. D. (2015). Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing (Volume 1: Long papers)* (pp. 1556–1566). Beijing, China: Association for Computational Linguistics, http://dx.doi.org/10.3115/v1/P15-1150.

Tang, Z., Hu, Q., Hu, Y., Kuang, W., & Chen, J. (2022). SeVulDet: A semantics-enhanced learnable vulnerability detector. In *2022 52nd Annual IEEE/IFIP international conference on dependable systems and networks* (pp. 150–162). http://dx.doi.org/10.1109/DSN53405.2022.00026.

Tang, Z., Shen, X., Li, C., Ge, J., Huang, L., Zhu, Z., et al. (2022). AST-trans: Code summarization with efficient tree-structured attention. In *Proceedings of the 44th international conference on software engineering* (pp. 150–162). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3510003.3510224.

Tian, Z., Tian, B., Lv, J., & Chen, L. (2023). Learning and fusing multi-view code representations for function vulnerability detection. *Electronics*, *12*(11).

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint arXiv:1710.10903.

Wang, S., Wang, J., Nam, J., & Nagappan, N. (2021). Continuous software bug prediction. In *Proceedings of the 15th ACM / IEEE international symposium on empirical software engineering and measurement*. New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3475716.3475790.

Wi, S., Woo, S., Whang, J. J., & Son, S. (2022). Hiddencpg: Large-scale vulnerable clone detection using subgraph isomorphism of code property graphs. In *Proceedings of the ACM web conference 2022* (pp. 755–766). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3485447.3512235.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(1), 4–24. http://dx.doi.org/10.1109/TNNLS.2020.2978386.

Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., & Jin, H. (2022). Vulcnn: An image-inspired scalable vulnerability detection system. In *2022 IEEE/ACM 44th international conference on software engineering* (pp. 2365–2376). http://dx.doi.org/10.1145/3510003.3510229.

Xiao, Y., Chen, B., Yu, C., Xu, Z., Yuan, Z., Li, F., et al. (2020). MVP: Detecting vulnerabilities using Patch-Enhanced vulnerability signatures. In *29th USENIX security symposium* (pp. 1165–1182). USENIX Association.

Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. (2014). Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE symposium on security and privacy* (pp. 590–604). http://dx.doi.org/10.1109/SP.2014.44.

Yamaguchi, F., Lottmann, M., & Rieck, K. (2012). Generalized vulnerability extrapolation using abstract syntax trees. In *Proceedings of the 28th annual computer security applications conference* (pp. 359–368). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2420950.2421003.

Yang, Z., Yang, D., Dyer, C., He, X., Smola, A., & Hovy, E. (2016). Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: Human language technologies* (pp. 1480–1489). San Diego, California: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/N16-1174.

Younis, A., Malaiya, Y., Anderson, C., & Ray, I. (2016). To fear or not to fear that is the question: Code characteristics of a vulnerable functionwith an existing exploit. In *Proceedings of the sixth ACM conference on data and application security and privacy* (pp. 97–104). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2857705.2857750.

Yu, N., Yuzhe, O., Michael, P., Feng, C., & Haipeng, C. (2023). VULGEN: Realistic vulnerability generation via pattern mining and deep learning. In *Proceedings of the 2023 IEEE/ACM international conference on software engineering* (p. 1).

Zhang, J., Wang, X., Zhang, H., Sun, H., & Liu, X. (2020). Retrieval-based neural source code summarization. In *2020 IEEE/ACM 42nd international conference on software engineering* (pp. 1385–1397).

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., & Liu, X. (2019). A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st international conference on software engineering* (pp. 783–794). http://dx.doi.org/10.1109/ICSE.2019.00086.

Zheng, Y., Pujar, S., Lewis, B., Buratti, L., Epstein, E., Yang, B., et al. (2021). D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. In *ICSE-SEIP '21*, *Proceedings of the 43rd international conference on software engineering: software engineering in practice* (pp. 111–120). IEEE Press, http://dx.doi.org/10.1109/ICSE-SEIP52600.2021.00020.

Zhou, Y., Liu, S., Siow, J., Du, X., & Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Proceedings of the 33rd international conference on neural information processing systems*. Red Hook, NY, USA: Curran Associates Inc..

Zou, D., Wang, S., Xu, S., Li, Z., & Jin, H. (2021). MuVulDeePecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable and Secure Computing*, *18*(5), 2224–2236. http://dx.doi.org/10.1109/TDSC.2019.2942930.